Scheduling Logic

Arrival — how a trip becomes a dated study plan

Kevin Francisco · September 2026 · Design specification

The problem

Spaced repetition systems optimize for indefinite retention. They schedule reviews at expanding intervals so a learner remembers an item forever, with no notion of when the knowledge is needed.

A traveler needs the opposite. They need maximum recall on a specific set of dates and do not care what happens after. Arrival treats the plan as a constrained optimization against a deadline: given the days remaining and the minutes available per day, choose the highest-value items the user can realistically absorb, and time the reviews so retention peaks on the trip.

Inputs

Input	Source	Used for
destination_city	Intake	Content selection, city specificity weighting
trip_start, trip_end	Intake	Countdown, review clamp, trip-length modifier
today	System	days_until
activity_tags	Intake	Item ranking by base utility
minutes_per_day	Intake	Capacity
prior_level	Intake	Starting difficulty, skip rules

Step 1 — Sizing the plan

```
days_until = trip_start − today

capacity = days_until × minutes_per_day

plan_size = clamp(capacity × LEARN_RATE, 40, 180)
```

LEARN_RATE is a tunable constant representing new items acquired per minute of study. Starting value 0.6, to be calibrated against observed completion data. The floor of 40 exists because a plan below that is not worth generating; the ceiling of 180 exists because beyond it the product stops being finite and starts being a course.

Trip-length modifier. Trips under 7 days weight transactional, single-exchange items. Trips over 14 days add weight to repeated-interaction and social items, since the traveler will face the same hosts and counter staff more than once.

Step 2 — Ranking the items

Each candidate item carries four signals:

- **base_utility** (0–100) per activity tag
- **regret_weight** — how many travelers to this city flagged the item as something they wished they had known
- **stakes_flag** — emergency, medical, allergy, and dietary items bypass ranking entirely
- **city_specificity** — city-level items outrank country-level generics when the city matches

Safety floor. Twelve items are force-included in every plan regardless of plan size or activity tags, covering emergency, medical, allergy, and “I am lost” phrasing. The floor is filled first, then the remaining slots are filled by rank.

Transparency. Every item exposes its ranking rationale in the UI. The product's credibility rests on the user believing the prioritization, so the reasoning is never hidden.

Step 3 — Scheduling the reviews

Modified SM-2 spaced repetition with one change that defines the product:

> *The final scheduled review of every item is clamped to fall within 3 days of departure.*

Standard SM-2 lets intervals expand indefinitely, which means an item learned early in a long runway may not resurface for weeks and will be cold on the trip. Clamping the terminal interval forces every item back through the queue in the final window, so the whole deck peaks together on the dates that matter.

Short-runway fallback. If days_until < 7, spacing is abandoned entirely. The plan collapses to massed practice on the safety floor plus the top 25 ranked items. Spaced repetition needs runway; below a week it produces worse outcomes than concentrated drilling.

Step 4 — Re-fitting

The plan is not fixed at generation. Two events trigger a recompute:

Falling behind

If actual completion trails the schedule, the app offers to shrink and re-rank rather than letting the user fail against the original plan. The safety floor is never cut. Prompt: “You're 6 days out and 40% through. Want me to cut this to the 30 items that matter most?”

Learning ahead

The user decides their own capacity and is never hard-blocked. Extra sessions pull the next items from the ranked queue in plan order. After each extra session the schedule is redistributed across the remaining days, since pulling items forward compresses the runway and would otherwise leave empty days at the end. The review clamp is preserved through every recompute.

Worked example

	Traveler A	Traveler B
Destination	Kyoto, 7 days	Kyoto, 30 days
Runway	23 days	180 days
Minutes per day	10	10
Capacity	230 minutes	1,800 minutes
Raw plan size	138	1,080
After clamp	138	180 (ceiling)
Weighting	Transactional items favored	Repeated-interaction and social items added
Terminal reviews	Clustered days 20–23	Clustered days 177–180

Same city, same daily commitment, materially different plans. That contrast is the demonstration the product is built around.

Calibration and open items

- LEARN_RATE is a guess until there is completion data. It is the single highest-leverage constant in the system.
- The 3-day clamp window is untested. Too tight and the final days become overloaded; too loose and the peak drifts before departure.
- The 40 and 180 bounds are judgment calls about what feels finite.
- No decay model yet for items learned and then re-fit out of a plan.
- Whether repeated re-fits should have a floor beyond the 12 safety items is unresolved.

This document describes the intended design. Where the shipped implementation differs, the implementation is the source of truth and this spec should be updated to match.